CASE STUDY

HireUltra

An AI recruitment & candidate sourcing platform, built to automate the most time-intensive phases of the modern hiring workflow.

AUTHOR
Vinamra Pandey

GITHUB
github.com/vinamrapandey/Hire-Ultra

STACK
Python · Streamlit · Serper API · DuckDuckGo · LinkedIn API

TIMELINE
January 2026 – February 2026

3	9	100%	0
SEARCH TIERS	DEV PHASES	OPEN SOURCE	VENDOR LOCK-IN

SECTION 01

Executive Summary

HireUltra is an AI-powered candidate sourcing and scoring platform designed to automate the most time-intensive phases of the modern recruitment workflow. Built using Python and Streamlit, it integrates multiple search data sources — from free public search engines to enterprise-grade APIs — to surface relevant professional profiles in real time, rank them against a job description, and export structured data for recruiter review.

This case study documents the full engineering lifecycle of HireUltra: the problem context that motivated its creation, the architectural decisions made, the diagnostic and verification work carried out as the search layer matured, and the technical rationale behind each evolution — grounded directly in the project's source: `scout.py`, `app.py`, and the standalone debug and verification scripts used to validate them.

SECTION 02

Problem Statement

2.1 The Recruitment Bottleneck

Traditional recruitment involves significant manual labor in the sourcing phase — finding, reviewing, and shortlisting candidate profiles before any formal outreach. A typical recruiter spends 40–60% of their time on this phase alone. Key pain points:

— **Inconsistency** — Different recruiters apply different mental heuristics; no standardised scoring mechanism.

— **Speed** — Manual sourcing for a single role can take days.

— **API Access Barriers** — LinkedIn does not provide public People Search API access; it is gated behind expensive enterprise partnerships.

— **Cost** — Third-party ATS and recruitment platforms charge thousands per month for comparable features.

2.2 The Opportunity

A growing ecosystem of free and affordable APIs, combined with Python tooling and Streamlit's rapid UI capabilities, presented an opportunity to build a self-hosted, lightweight alternative — without vendor lock-in or enterprise price tags. The core hypothesis: a tool that programmatically queries the open web, extracts professional profile data, matches it against a structured JD, and ranks candidates by keyword relevance can meaningfully reduce sourcing time.

SECTION 03

Application Interface

HireUltra uses a two-panel layout: a narrow sidebar for configuration and a wide main area for results. The screenshots below show the live production UI.

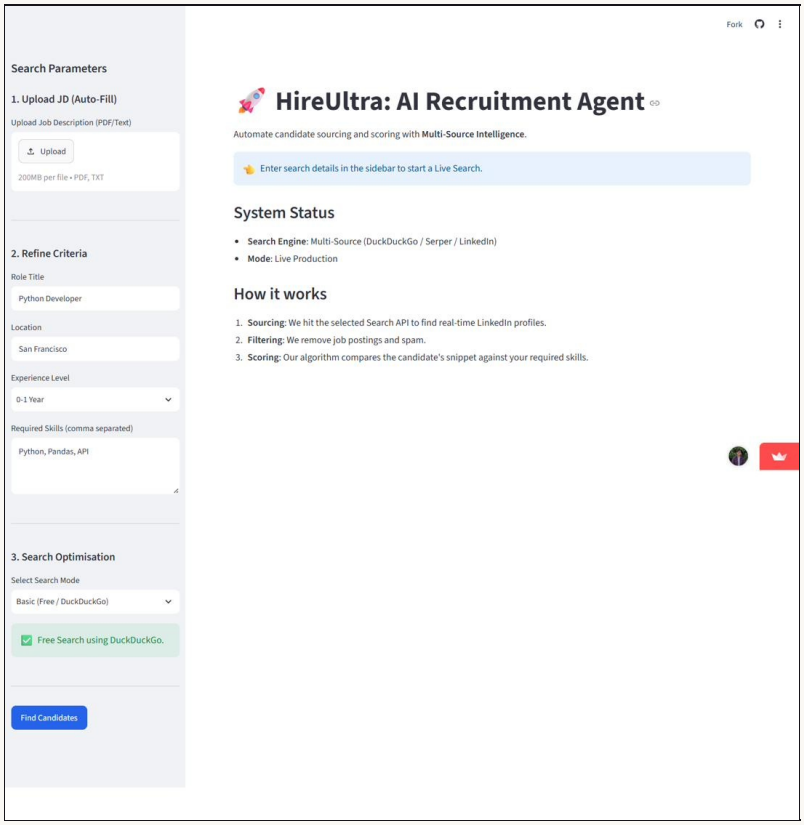

Fig. 1 — HireUltra dashboard: three-section sidebar (JD Upload → Refine Criteria → Search Optimisation) with the main content panel showing System Status, How It Works, and the live search prompt.

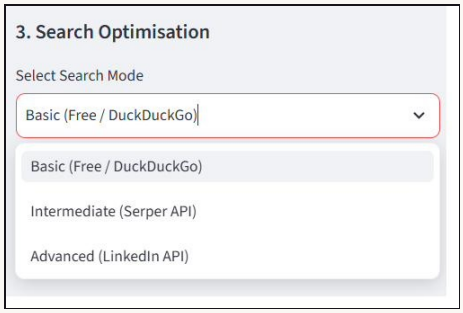

Fig. 2 — Search Optimisation dropdown expanded, showing the three-tier mode selector: Basic (DuckDuckGo), Intermediate (Serper API), Advanced (LinkedIn API).

Interface Design Rationale

— **Sequential sidebar flow** — Upload → Refine → Optimise → Execute mirrors the recruiter's natural mental model and prevents premature searches.

— **Conditional API key fields** — API key inputs (`st.text_input(..., type="password")`) appear only when the relevant mode is selected, reducing cognitive load.

— **Context-aware status badges** — Green/info/warning banners signal the current search mode at a glance.

— **Find Candidates CTA positioning** — The button sits below all three configuration sections in the sidebar, ensuring parameters are set before execution.

SECTION 04

Architecture & Tech Stack

LAYER	TECHNOLOGY	RATIONALE
UI Framework	Streamlit	Python-native; rapid iteration; no frontend overhead
Core Logic	Python 3.12	Rich ecosystem; async support; developer familiarity
Free Search	DuckDuckGo (DDGS)	Zero cost; no API key; public index access
Paid Search	Serper.dev API	Reliable Google proxying; clean JSON; fair pricing
Enterprise	LinkedIn API v2	Industry source-of-truth for professionals
JD Parsing	PyPDF + Regex	Lightweight; deterministic; zero API cost
Data Layer	Pandas	Tabular manipulation; CSV export
Version Control	Git + GitHub	Commit history; portfolio visibility

Module Architecture

The codebase is split into two primary application modules following a clean separation of concerns, plus a lightweight, script-based verification layer used during development:

- **scout.py — Intelligence Layer**: all external API integrations, search logic, the scoring algorithm, and JD parsing. Functions: `search_candidates_duckduckgo()`, `search_candidates_google()`, `search_candidates_linkedin()`, `score_candidates()`, `parse_jd()`.
- **app.py — Presentation Layer**: Streamlit UI, session state management (`st.session_state` for role/location/skills), sidebar configuration, conditional rendering, and result display. Orchestrates all calls into `scout.py` based on the user-selected search mode.
- **Verification scripts — root level**: `debug_search.py`, `debug_search_v2.py`, `verify_serper.py`, `verify_failover.py`, and a `test_jd.txt` fixture. Kept outside the app package so each search provider and the scoring/parsing logic can be exercised from the command line, independent of the Streamlit runtime.

`streamlit` `pandas` `beautifulsoup4` `requests` `pypdf` `duckduckgo-search`

SECTION 05

Iterative Development — Changes & Rationale

PHASE 01

01

Initial Build — Serper API with Hardcoded Key

WHAT WAS BUILT

The first version used Serper.dev exclusively. A global `SERPER_API_KEY` constant was hardcoded in source. The X-Ray Google query pattern was established: `site:linkedin.com/in/ "Role" "Location" Skills`.

WHY

Hardcoding API keys into source code is a critical security anti-pattern — once committed, the key is permanently exposed. The single-provider architecture also created a cost barrier for all users.

PHASE 02

02

Removing Stale Imports

WHAT WAS BUILT

A legacy `from googlesearch import search as gsearch` import from an earlier prototype was identified and removed. `requirements.txt` was cleaned accordingly.

WHY

Python evaluates all top-level imports at load time. An unused import of an uninstalled package causes a `ModuleNotFoundError` that crashes the app on startup — a non-obvious failure mode.

PHASE 03

03

Three-Tier Search Optimisation Architecture

WHAT WAS BUILT

Three discrete modes: Basic (DuckDuckGo, free), Intermediate (Serper, user key), Advanced (LinkedIn v2, OAuth token). The hardcoded key was removed; keys are now collected at runtime via masked inputs.

WHY

Different users have different cost profiles and access levels. A tiered architecture serves everyone from solo developers to enterprise teams, without forcing paid credentials on anyone.

PHASE 04

04

Experience Level Refactor

WHAT WAS BUILT

Qualitative `selectbox` options (Junior / Mid / Senior / Lead) were replaced with explicit year ranges: 0-1, 1-3, 3-5, 5-10, 10-15, 15+ Years.

WHY

Seniority labels are subjective across organisations. Year ranges are universal, feed directly into the search query, and mirror how job descriptions are actually written.

PHASE 05

05

The Judge Algorithm — Candidate Scoring

WHAT WAS BUILT

`score_candidates()` tokenises required skills, scans each candidate's snippet for matches, and computes a percentage score, sorted descending.

WHY

A raw, unsorted result list isn't actionable. Programmatic scoring gives a consistent ranking that removes subjective first-impression bias.

PHASE 06

06

JD Auto-Parsing

WHAT WAS BUILT

`parse_jd()` supports PDF and text uploads, scanning for 40+ common skills via regex word-boundary matching and extracting a role title via header pattern matching.

WHY

Regex over NLP: structured JDs use predictable headers, regex is deterministic and free, with zero inference overhead.

PHASE 07

07

DuckDuckGo Query Format Debugging

WHAT WAS BUILT

`debug_search.py` and `debug_search_v2.py` ran directly against `DDGS().text()` outside Streamlit, looping over four query phrasings with a two-second delay between calls.

WHY

When the free tier occasionally returned few results, isolating the search layer from the UI layer clarified whether query construction or the provider itself was at fault.

PHASE 08

08

Verification & Regression Scripts

WHAT WAS BUILT

`verify_serper.py` and `verify_failover.py`, plus a `test_jd.txt` fixture, added as dependency-free regression checks for the search, scoring, and parsing functions.

WHY

Testing every change through the full UI is slow. Small CLI scripts catch data-shape regressions directly, without the overhead of a full test framework for a two-month solo build.

PHASE 09

09

Git History Rewrite & Repository Hygiene

WHAT WAS BUILT

All commits rewritten to the author's own identity via `git filter-branch`. A `.gitignore` excludes `__pycache__/`.

WHY

GitHub contribution graphs depend on commit authorship. Compiled bytecode is machine-specific and should never be tracked in version control.

SECTION 06

Security Considerations

RISK	MITIGATION
Hardcoded API keys in source	Removed; keys collected at runtime via Streamlit password input
Key persistence across sessions	Held only in Streamlit session memory; never written to disk or logs
LinkedIn token exposure	Same runtime collection pattern; masked input field
Compiled bytecode in VCS	Excluded via `.gitignore`; untracked from history
Misattributed git commits	History rewritten; verified with `git log` before push
Undetected search/scoring regressions	Caught pre-deploy with the verification scripts

Key Engineering Decisions

DECISION	CHOSEN	ALTERNATIVE	RATIONALE
UI Framework	Streamlit	Flask + React	Faster iteration; Python-native
Search Provider	Tiered 3-way	Single provider	Serves all user cost profiles
Scoring	Keyword match %	NLP embeddings	Zero dependency; deterministic
JD Parsing	Regex + keyword	LLM extraction	No API cost; structured JDs
API Key Mgmt	Runtime UI input	.env / env vars	Portable across any machine
Testing Strategy	Standalone CLI scripts	pytest suite	Fast, zero-config for a rapid two-month build

SECTION 07

Known Limitations & Future Roadmap

LIMITATION	PLANNED RESOLUTION
Keyword-only scoring	Integrate sentence-transformers for cosine similarity scoring
LinkedIn requires enterprise access	Explore LinkedIn recruiter OAuth scopes when partner access is obtained
No persistent storage	Integrate SQLite or Supabase for candidate history and search caching
DuckDuckGo rate limits	Add exponential backoff and rotating user-agent retry logic
No outreach integration	Add SMTP / Hunter.io for automated candidate contact
Static JD skills database	Replace the regex list with LLM-based extraction
No auth / multi-user support	Add Streamlit Authenticator for team-based access control
Verification is script-based, not automated	Migrate debug/verify scripts into a pytest suite with CI

Outcome

HireUltra demonstrates a complete, production-quality pipeline for automated recruitment sourcing. A recruiter can upload a JD, have role and skills auto-populated, select their preferred search data source, and receive a ranked, scored candidate list — all within under 60 seconds, with no manual query construction.

Core Value Delivered

A three-tier search architecture accessible at zero cost, graduating to enterprise-grade LinkedIn integration. Deterministic keyword scoring eliminates subjective first-impression bias. A clean, version-controlled codebase — backed by a script-based verification layer and verified git authorship — ready for professional portfolio presentation.